norma.ai — Complete Technical Wiki

> **Purpose:** Everything you need to know to answer any question about this platform during a presentation. Covers architecture, every concept, every engine, the frontend, agents, APIs, and how it all fits together.

Table of Contents

1. What is norma.ai?
2. How to Run It
3. System Architecture Overview
4. Key Concepts Glossary
5. The Contract
6. Trust Score & Tier System
7. Enforcement Engine
8. Quality Scoring
9. Observability: Spans, Traces & OpenTelemetry
10. Compliance Framework
11. Attribution Engine
12. Enhancement Engine
13. Context Router
14. Database Models
15. API Reference
16. Integration Layer (How Agents Connect)
17. norma-watch CLI
18. Frontend Dashboard
19. Demo Agents
20. Real-Time Events (SSE)
21. Multi-Agent Orchestration
22. Contract Lifecycle (End-to-End)
23. Agent Execution Flow (End-to-End)
24. Compliance Posture
25. Q&A Engine
26. Feature Flags
27. Security
28. File Map (Quick Reference)
29. What's Real vs. Planned
30. Common Questions & Answers

---

1. What is norma.ai?

One sentence: norma.ai is an operating system for AI agents — it tells you what agents are allowed to do, enforces those boundaries in real time, and gives you evidence of whether they're behaving.

The problem it solves: When you deploy LLM-powered agents in production, you lose visibility and control. Agents can call tools they shouldn't, access data they're not authorized to read, produce outputs with PII, or spiral in cost. norma.ai wraps every agent with a governance layer that enforces policy, records every action as a trace, scores quality, builds trust over time, and surfaces compliance findings — all without changing the agent's code.

What it is NOT: norma is not the agent itself. It's the layer around the agent. The agent (LangChain, LangGraph, OpenAI, etc.) runs its own logic; norma intercepts every tool call, checks it against a contract, records a span, and updates the agent's trust score.

Who uses it:

- **VPs / Executives:** See which agents are trusted, which are violating policy, compliance status, cost trends. Business-friendly messages.
- **Engineers:** Full trace trees, span-level debugging, token costs, quality breakdown, enforcement details.

---

2. How to Run It

```
# 1. Backend (from /backend)
poetry run uvicorn norma.main:app --host 0.0.0.0 --port 8080 --reload

# 2. Seed demo data (first time or before a demo)
cd backend && poetry run python -m norma.seed

# 3. Frontend (from /frontend)
npm run dev      # runs on port 3030

# 4. Run a real monitored agent (requires OPENAI_API_KEY)
export OPENAI_API_KEY=sk-...
cd backend && poetry run norma-watch --agent-file
../agents/financial_reader/earnings_report_reader.py

# 5. Run all agents (from project root)
./scripts/run_all_agents.sh --remote   # streams to dashboard

# 6. Run tests
cd backend && poetry run pytest tests/ -v   # 97 passing
```

URLs:

- API docs (Swagger): `http://localhost:8080/docs`
- Dashboard: `http://localhost:3030`

---

3. System Architecture Overview

```
┌──────────────────────────────────────────────────────────────────────┐
│                        norma.ai Platform                             │
│                                                                      │
│                                                                      │
│  ┌──────────────┐      ┌──────────────────────────────────────┐     │
│  │   Frontend   │      │            Backend (FastAPI)         │     │
│  │  Next.js 14  │◄─────│                                      │     │
│  │  React 19    │      │  ┌──────────┐   ┌──────────────────┐ │     │
│  │  TypeScript  │      │  │ API Layer│   │  Core Engines    │ │     │
│  │  Recharts    │      │  │(10 routes│   │  - Trust         │ │     │
│  └──────────────┘      │  │  40+ ep) │   │  - Enforcement   │ │     │
│                        │  └──────────┘   │  - Quality       │ │     │
│                        │       │         │  - Compliance    │ │     │
│  ┌──────────────┐      │  ┌────▼─────┐   │  - Attribution   │ │     │
│  │  Real Agent  │      │  │   ORM    │   │  - Enhancement   │ │     │
│  │  (LangChain/ │      │  │(SQLAlch) │   └──────────────────┘ │     │
│  │  LangGraph/  │      │  └──────────┘                        │     │
│  │  OpenAI)     │      │       │                              │     │
│  └──────────────┘      │  ┌────▼──────────────────┐           │     │
│        │               │  │  SQLite (dev)         │           │     │
│  ┌────▼─────────┐      │  │  PostgreSQL (prod)    │           │     │
│  │ NormaSession │      │  └───────────────────────┘           │     │
│  │ (middleware) │      │                                      │     │
│  └──────────────┘      └──────────────────────────────────────┘     │
└──────────────────────────────────────────────────────────────────────┘
```

Technology Stack:

Layer	Technology
Backend	Python 3.11, FastAPI, SQLAlchemy async, Pydantic v2
Database	SQLite (dev), PostgreSQL (prod) — 16 ORM tables
Frontend	Next.js 14, React 19, TypeScript, Tailwind CSS, Recharts
LLM Integration	OpenAI GPT-4o/4o-mini, LangChain ≥0.3, LangGraph ≥0.2, OpenAI Agents SDK
Compliance	OWASP LLM Top 10, NIST AI RMF, EU AI Act
Real-time	Server-Sent Events (SSE)
Observability	Custom OpenTelemetry-compatible span system

---

4. Key Concepts Glossary

Agent

A Python program (LangChain, LangGraph, OpenAI SDK, or custom) that uses an LLM to perform tasks via tools. Norma wraps it — it does not replace it.

Contract

A YAML policy document that governs one agent. Defines: what tools it can use, what data it can access, SLA limits (cost/latency/quality), output patterns to block (PII), and the trust score rules. Every agent must have an active contract before it can run under norma.

Run

One end-to-end execution of an agent. Captures tokens, cost, latency, quality score, all spans (trace), violations, and the trust score after. Runs form a tree for multi-agent orchestration (parent run → child runs).

Span

An OpenTelemetry-compatible record of one operation within a run. A run = one or many spans. Types: `llm_call`, `tool_call`, `agent_handoff`, `enforcement_check`, `guardrail`, `session`. Each span has: start/end time, input/output (truncated), token counts, cost, model name.

Violation

A recorded event when enforcement blocked (or should have blocked) an action. Has: policy rule, what was attempted, whether it was blocked, event type.

Trust Score

A float 0.0–1.0 representing how reliably the agent has followed its contract. Increases +0.025 per clean run. Drops −0.25 on any violation. Never auto-promotes — a human must approve tier upgrades.

Tier

Derived from trust score: **restricted** (default, 0.0–0.64), **standard** (0.65+, 10 clean runs), **trusted** (0.82+, 20 clean runs). Controls enforcement strictness and visible privilege.

Enforcement

The five deterministic checks that run before every tool call. If any check fails, the tool is blocked and a violation is recorded. Enforcement is always on; it cannot be disabled per-call.

Quality Score

A 0.0–1.0 score assigned to each run. Computed from: output length check, error keyword detection, format compliance, contract scope check, plus an optional LLM-as-judge score from GPT-4o-mini.

Session

Groups related runs together (e.g., a multi-turn conversation). Shared context is tracked across runs in the same session.

Onboarding

The process of registering an agent with norma. norma scans the agent's Python file via AST to discover `@tool`-decorated functions, generates a contract proposal, and waits for human approval.

Attribution

When a multi-agent workflow fails, norma uses probabilistic scoring to identify which node (sub-agent) is most likely responsible, with a confidence score and evidence.

Enhancement

A data-driven recommendation to improve the agent's workflow — e.g., "this tool wastes 80% of its tokens" or "this tool has been blocked 5 times; harden the contract."

---

5. The Contract

What it is

A YAML document that is the source of truth for what an agent is allowed to do. Contracts are versioned, human-approved, and stored in the database. Only one contract version is active at a time per agent.

Full YAML Structure

```yaml
agent_id: financial-reader-v1
version: "1.0"
tier: restricted

scope:
  description: "Read and summarize quarterly earnings from public
directory"
  allowed_tasks: [document_summary, data_extraction, trend_analysis]

authorities:
  tools:
    allow: [list_reports, read_report, text_analysis]
    deny:  [read_confidential, web_search, email_sender, external_api]
  data:
    allow: [data/public/**]
    deny:  [data/confidential/**, data/internal/**]

output_constraints:
  deny_patterns: [pii_regex, credential_regex, credit_card_regex]

sla:
  max_cost_per_run: 0.50      # USD
  max_latency_seconds: 30
  min_quality_score: 0.80

trust:
  initial_score: 0.40
  tier_thresholds:
    standard: {min_score: 0.65, min_clean_runs: 10}
    trusted:  {min_score: 0.82, min_clean_runs: 20}
  violation_penalty: 0.25
  clean_run_increment: 0.025
```

```yaml
delegation:                    # optional, for orchestrators
  subagents:
    - id: fetcher-agent
      context_limit_tokens: 2000
    - id: analyzer-agent
      context_limit_tokens: 3000
```

Contract Versioning

- Every change to a contract creates a new version record (`ContractVersion` model)
- Versions record: who changed it, who approved it, reason, diff vs previous, timestamp
- `is_active=True` on only one version per agent at a time
- Pending upgrades sit in `Agent.pending_contract_version` until human approval

Where contracts live in code

- `backend/norma/core/contract_engine.py` — parsing, validation, summaries
- `backend/norma/core/contract_generator.py` — auto-generation (LLM or stub)
- `backend/norma/api/contracts.py` — REST endpoints
- `backend/norma/models/contract.py` — ORM models

Human Approval Requirement

Contracts are **never auto-activated**. This is a design principle. The trust engine can propose a new contract version, but a human must call `POST /api/contracts/{agent_id}/activate` for it to take effect.

---

6. Trust Score & Tier System

How it works

Every agent starts at trust score **0.40** (restricted tier). Trust changes on every run:

Event	Change
Clean run (no violations)	+0.025
Any violation	−0.25
Tier revocation (on violation)	Back to "restricted"

Tier Thresholds

Tier	Min Score	Min Clean Runs	Notes
restricted	—	—	Default. Most locked down
standard	0.65	10	Proposed after meeting threshold

Tier	Min Score	Min Clean Runs	Notes
trusted	0.82	20	Proposed after meeting threshold

Important: Reaching the threshold only **proposes** a new contract version. A human must approve it before the tier upgrade takes effect.

Trust is permanent evidence

Trust score is recorded on every Run (`trust_score_after` field). You can see the full trust trajectory on the agent detail page (TrustSparkline).

Code location

`backend/norma/core/trust_engine.py` — `TrustState` dataclass, `record_clean_run()`, `record_violation()`, `approve_pending_contract()`

---

7. Enforcement Engine

What it is

Five deterministic checks that run **before every tool call**. If any check fails, the tool is not executed, a violation is recorded, and a block message is returned to the agent.

The Five Checks (in order)

1. **Tool Access** (`_check_tool_access`)
   - Checks the requested tool name against `authorities.tools.deny` (deny wins) and `authorities.tools.allow`
   - Example block: agent tries to call `read_confidential` → blocked by deny list
2. **Data Path** (`_check_data_path`)
   - Checks the data path argument against `authorities.data.deny` (glob pattern matching via `fnmatch`)
   - Example block: agent passes `data/confidential/exec_comp.txt` → matches `data/confidential/**` → blocked
3. **Output Patterns** (`_check_output_patterns`)
   - Regex scan of the proposed output for PII patterns:
     - `credit_card_regex`: `\b(?:\d[ -]?){13,16}\b`
     - `ssn`: `\b\d{3}[- ]\d{2}[- ]\d{4}\b`
     - `pii` (name pattern): `\b[A-Z][a-z]+ [A-Z][a-z]+\b`
     - `credential`: `(?i)(password|api_key|secret|token)\s*[:=]\s*\S+`
4. **Cost SLA** (`_check_cost_sla`)
   - Checks cumulative cost so far in this run against `sla.max_cost_per_run`

5. **Latency SLA** (`_check_latency_sla`)
    - Checks elapsed time so far against `sla.max_latency_seconds`

Design principles

- **Deterministic:** No LLM calls. No false positives from ML.
- **Fast:** Runs in microseconds (regex + dict lookup).
- **Sequential:** First failing check stops evaluation immediately.
- **Cannot be bypassed:** Enforcement is applied by the session wrapper before any tool executes.

Code location

`backend/norma/core/enforcement.py` — `enforce(context, contract)` → `EnforcementResult`

---

8. Quality Scoring

What it is

A 0.0–1.0 score assigned to a run's output. Composed of deterministic checks plus an optional LLM judge.

Deterministic Checks (always run)

Check	What it measures	Scoring
`output_length`	Is the output substantial?	empty=0.0, <20=0.2, <50=0.4, <100=0.6, <200=0.8, ≥200=1.0
`error_keywords`	Does output contain failure signals?	penalizes "error", "failed", "unable", etc.
`format_compliance`	Is output in the expected format?	JSON parseable, or markdown headers/lists present
`contract_scope`	Does output leak denied patterns?	PII/credential regex check

LLM-as-Judge (optional, requires `OPENAI_API_KEY`)

When `enable_llm_quality_scoring=True` (default), GPT-4o-mini evaluates the output on:

- Relevance to the task
- Completeness
- Accuracy
- Format adherence

Returns a score + rationale text (stored in `run.quality_rationale`).

Composite Score

```
final_score = 0.4 × deterministic_score + 0.6 × llm_score   (if LLM
enabled)
final_score = deterministic_score                            (if LLM
disabled)
```

Quality breakdown storage

`run.quality_breakdown` — JSON object with per-check scores:

```
{
  "output_length": 0.8,
  "error_keywords": 1.0,
  "format_compliance": 0.5,
  "contract_scope": 1.0
}
```

Code location

`backend/norma/core/quality_scorer.py` — `evaluate_quality()` (async), `evaluate_quality_sync()` (deterministic only)

---

9. Observability: Spans, Traces & OpenTelemetry

What is OpenTelemetry (OTel)?

OpenTelemetry is an open standard for collecting telemetry data (traces, metrics, logs) from software. It uses the concept of **spans** — records of individual operations — which form a **trace** (a tree of spans representing a complete request).

norma implements an **OTel-compatible span system** in pure Python without using the external OpenTelemetry SDK. This means the span data structure and naming follows OTel conventions, but it's collected in-process and persisted to the database.

What is a Span in norma?

A span = one logical operation recorded during a run. Each span captures:

- `span_id`: 16-character hex identifier (OTel-compatible)
- `span_type`: what kind of operation (`llm_call`, `tool_call`, `agent_handoff`, `enforcement_check`, `guardrail`, `session`)
- `name`: tool name, model name, or agent name
- `parent_span_id`: for nested operations (creates a tree)
- `status`: `ok`, `error`, or `blocked`
- `start_time` / `end_time`: precise timing
- `input_data` / `output_data`: truncated JSON of what went in and came out
- `tokens_in`, `tokens_out`, `cost_usd`: LLM metrics

- `model_name`: e.g., `gpt-4o`, `gpt-4o-mini`
- `attributes`: JSON metadata (temperature, max_tokens, prompt_hash, quality_subscore, etc.)

What is a Trace?

In norma, a trace = all the spans for one Run. The `trace_id` on each span is the Run's database ID. You can see the full trace tree at `GET /api/runs/{run_id}/spans`.

Span Types Explained

Type	When created	What it records
`session`	When NormaAgentSession context starts	Overall run wrapper
`llm_call`	Each LLM invocation	Model, tokens, cost, prompt hash
`tool_call`	Each tool execution attempt	Tool name, input args, output
`enforcement_check`	Each enforcement decision	Policy rule, allowed/blocked
`agent_handoff`	Sub-agent delegation	Which sub-agent, context tokens
`guardrail`	Output pattern check	Pattern matched, blocked/allowed

How spans are collected

`backend/norma/core/trace.py` — `TraceCollector` class:

- `start_span()` → creates `SpanData` with start timestamp
- `end_span()` → fills in output, timing, metrics
- At session exit, all spans are batch-persisted to the `spans` table

OTLP Export (planned)

Feature flag `enable_otlp_export=True` would send spans to an external OTel collector (Jaeger, Tempo, etc.) via OTLP gRPC at `otlp_endpoint`. Not yet implemented.

Viewing spans in the UI

Run detail page (`/runs/{id}`) shows:

- **SpanTree**: hierarchical collapsible tree of all spans
- **WaterfallTimeline**: Gantt-style view of span timing
- **PromptInspector**: message-level breakdown for LLM spans

---

10. Compliance Framework

Three Standards Implemented

Standard	What it checks
OWASP LLM Top 10	The 10 most critical AI security risks

Standard	What it checks
NIST AI RMF	US AI Risk Management Framework
EU AI Act	European AI regulation requirements

OWASP LLM Top 10 Rules (implemented)

Rule ID	Name	What triggers it
LLM01	Prompt Injection	Spans contain "ignore previous instructions", "system prompt", "jailbreak"
LLM02	Insecure Output	Output contains `<script>`, `javascript:`, `DROP TABLE`
LLM06	Sensitive Information Disclosure	Output matches SSN, credit card, or credential patterns
LLM08	Excessive Agency	Tool call count > 12 in a run
LLM09	Overreliance	Run succeeded but quality_score < 0.5

NIST AI RMF Rules

Three basic checks (Map, Measure, Manage) — currently placeholder implementations validating logging and risk tracking.

EU AI Act Rules

- **Art. 17**: Risk management documentation
- **Art. 18**: Logging and audit trail
- **Art. 19**: Transparency requirements

Model Drift Rule

Detects when the agent's model name or contract version has changed between runs, flagging potential uncontrolled updates.

How compliance is evaluated

```
POST /api/compliance/evaluate  →  ComplianceEngine.evaluate(ctx)
                                        ↓
                                  12 rules run in parallel
                                        ↓
                              ComplianceResult (list of findings)
```

Each finding: `rule_id`, `passed` (bool), `details`, `evidence`.

Compliance Posture

`GET /api/compliance/{agent_id}/posture` — summary of pass/fail per standard. Shown as CMP badge on AgentCard in dashboard.

PDF Export

`GET /api/compliance/{agent_id}/export/pdf` — generates a PDF compliance report (uses fpdf2).

Code location

`backend/norma/core/compliance/` — `engine.py`, `owasp.py`, `nist.py`, `eu_ai_act.py`, `drift.py`, `rule.py`, `result.py`

---

11. Attribution Engine

The problem it solves

In a multi-agent pipeline (orchestrator → fetcher → analyzer → writer), when the final output is poor, which node caused it? Attribution answers this probabilistically.

How it works

1. For each node in the run tree, scores are computed:
    - `input_quality_score`: quality of what that node received
    - `output_quality_score`: quality of what it produced
    - `confidence_score`: how certain norma is about this node's work
2. Attribution probability:

    ```
    quality_delta = input_quality - output_quality
    attribution_prob = min(quality_delta × 1.5 + (0.2 if confidence < 0.70
    else 0), 1.0)
    ```

3. If a node's input quality is already < 0.60, all downstream nodes are excluded from blame.
4. Returns:
    - `most_likely_node`: agent ID of the likely culprit
    - `confidence`: 0.0–1.0
    - `evidence`: explanation text
    - `alternative_hypotheses`: [{node, confidence}] for other candidates

Design principle

Attribution is **probabilistic, not a verdict**. The confidence score and alternative hypotheses are always shown. norma never asserts guilt with certainty.

Code location

`backend/norma/core/attribution.py` `backend/norma/api/attributions.py` — `GET /api/attributions/{run_id}`

---

12. Enhancement Engine

What it is

Analyzes run history across an agent and generates data-driven recommendations for improving the workflow or contract.

Three Recommendation Types

Type	Trigger Condition	Recommendation
`token_waste`	input_tokens ≥ 600 AND output ≤ 15% of input	Reduce context window or restructure prompts
`violation_pattern`	Same policy rule violated ≥ 2 times	Harden contract: add the tool/path to deny list explicitly
`cost_hotspot`	One tool step > 35% of total run cost	Optimize that tool or switch to cheaper model

Confidence Levels

- **high**: ≥ 8 data samples
- **medium**: ≥ 4 samples
- **low**: < 4 samples

Output

Each recommendation includes:

- Type + title
- Evidence (data that triggered it)
- Confidence level + explanation
- `suggested_action`: plain English fix
- `expected_outcome`: what improvement to expect
- YAML snippet for contract update (where applicable)

Code location

`backend/norma/core/enhancement.py` `backend/norma/api/agents.py` — `GET /api/agents/{agent_id}/enhancements`

---

13. Context Router

What it is

Intercepts sub-agent delegation calls in multi-agent workflows, applies routing rules from the parent's contract, and tracks how context tokens are being used.

What it measures

Metric	Description
`tokens_available`	Token budget allocated to this sub-agent
`tokens_sent`	Actual tokens in the context passed
`utilization_ratio`	tokens_sent / tokens_available (0.0–1.0)
`routing_rules_applied`	Whether parent's routing rules were enforced
`output_overlap_ratio`	N-gram similarity between parent and child outputs (approximation)

Token counting

Approximated as `len(context) / 4` characters per token. Not using tiktoken — Phase 4 TODO.

Code location

`backend/norma/core/context_router.py` `backend/norma/models/context_metric.py`

---

14. Database Models

16 ORM tables using SQLAlchemy async. SQLite in development, PostgreSQL in production.

Core Tables

`agents`

The registry of all monitored agents.

```
agent_id (PK)       name, type (single|orchestrator|subagent)
department, owner   organizational metadata
current_tier        restricted | standard | trusted
trust_score         0.0–1.0
clean_run_count     cumulative violation-free runs
entry_point         relative path to agent.py
file_hash           16-char SHA-256 prefix for change detection
agent_code_version  increments when hash changes
code_status         ok | changed | missing
enabled             pause/resume flag
parent_agent_id     for sub-agents: which orchestrator owns them
framework           langchain | openai_func | openai_agents | langgraph
```

`runs`

One record per agent execution.

```
id (PK, autoincrement)
agent_id (FK)           contract_version    session_id
parent_run_id (FK)      initiated_by        (user|api|orchestrator:<id>)
input_tokens            output_tokens       cost_usd        latency_ms
quality_score           quality_rationale   quality_breakdown (JSON)
trust_score_after       completion_status
(success|failed|timeout|escalated)
timestamp
```

spans

OpenTelemetry-compatible trace records.

```
id (PK)             span_id (16-char hex, unique)
trace_id (FK→runs)  parent_span_id
span_type
(llm_call|tool_call|agent_handoff|enforcement_check|guardrail|session)
name                status (ok|error|blocked)
start_time          end_time        latency_ms
input_data          output_data     (JSON strings, truncated)
tokens_in           tokens_out      cost_usd
model_name (indexed) attributes (JSON)
```

contracts

Policy documents.

```
id (PK)         agent_id (FK)       version     yaml_content (TEXT)
summary_text    is_active (bool)    created_by  approved_by
activated_at    created_at
```

contract_versions

Audit trail for changes.

```
id (PK)         contract_id (FK)    diff_json
changed_by      approved_by         reason      timestamp
```

violations

Policy enforcement events.

```
id (PK)          run_id (FK)          agent_id (FK)
policy_rule      action_attempted     blocked (bool)
event_type
(access_blocked|tier_revocation|access_revoked|output_blocked)
scope            (review metadata string)    timestamp
```

Supporting Tables

Table	Purpose
`outcomes`	Quality score breakdown per run (1-to-1 with runs)
`run_steps`	Legacy flat tool call log (kept for backward compatibility)
`context_metrics`	Token routing metrics per delegation
`attribution_reports`	Per-run failure root cause (1-to-1 with runs)
`budgets`	Per-agent cost/run limits (daily or monthly)
`compliance_reports`	Sentinel governance sweep results
`prompt_snapshots`	Full LLM message history per run
`shared_contexts`	Cross-run context flow records
`memory_store`	Topic-keyed conversation memory (TTL-based)
`recommendations`	Enhancement engine outputs

Schema Migrations

`database.py` runs `_apply_schema_migrations()` on startup. It adds missing nullable columns using SQLite `PRAGMA table_info`. Safe to run repeatedly.

---

15. API Reference

All routes under `http://localhost:8080`. See full Swagger docs at `/docs`.

Agents API (`/api/agents`)

Method	Path	Description
GET	`/api/agents`	Fleet list with metrics
GET	`/api/agents/{id}`	Agent detail (contract, runs, violations)
POST	`/api/agents/onboard`	Register agent from filesystem
POST	`/api/agents/{id}/execute`	Run agent from UI (step/full/llm modes)
GET	`/api/agents/{id}/check-changes`	Detect file hash changes

Method	Path	Description
PATCH	`/api/agents/{id}`	Pause or resume agent
DELETE	`/api/agents/{id}`	Deregister
GET	`/api/agents/{id}/enhancements`	Workflow improvement recommendations

Execute modes:

- `mode=step` (default): run the next tool in sequence (cycles by run count)
- `mode=full`: run all tools end-to-end, return per-step results
- `mode=llm`: invoke real LLM agent (requires OPENAI_API_KEY, uses `build_llm_agent()`)

Runs API (`/api/runs`)

Method	Path	Description
POST	`/api/runs/ingest`	Remote telemetry ingest (from norma-watch)
GET	`/api/runs`	List runs (paginated)
GET	`/api/runs/{id}`	Run detail with spans + violations
GET	`/api/runs/{id}/tree`	Run tree (parent-child for multi-agent)
GET	`/api/runs/{id}/spans`	Span tree (nested)
GET	`/api/runs/{id}/metrics`	Aggregated cost, latency, quality
GET	`/api/runs/{id}/prompts`	PromptSnapshot message history
GET	`/api/runs/{id}/context-metrics`	Context routing budgets

Violations API (`/api/violations`)

Method	Path	Description
GET	`/api/violations/`	Recent violations (optional agent filter)
GET	`/api/violations/{agent_id}/audit`	Full audit log
POST	`/api/violations/{id}/review`	Acknowledge or dismiss as false positive

Review decisions: `acknowledged`, `dismissed_false_positive`

Contracts API (`/api/contracts`)

Method	Path	Description
GET	`/api/contracts/{agent_id}`	Version history
POST	`/api/contracts/{agent_id}/generate`	Auto-generate proposal
POST	`/api/contracts/{agent_id}/activate`	Approve and activate a version

Method	Path	Description
GET	`/api/contracts/{agent_id}/compare`	Diff two versions
POST	`/api/contracts/{agent_id}/update`	Create manually

Analytics API (`/api/analytics`)

Method	Path	Description
GET	`/api/analytics/{agent_id}`	Window-based metrics (7d/30d/90d)
GET	`/api/analytics/{agent_id}/trends`	Time-series + version checkpoints
GET	`/api/analytics/compare`	Multi-agent comparison

Version checkpoints on trends: lines marking contract changes (purple) and model changes (orange, e.g., "gpt-4o").

Compliance API (`/api/compliance`)

Method	Path	Description
POST	`/api/compliance/evaluate`	Run all 12 compliance rules
GET	`/api/compliance/{agent_id}/posture`	Pass/fail summary
GET	`/api/compliance/{agent_id}/export/pdf`	Download compliance PDF

Q&A API (`/api/qa`)

Method	Path	Description
POST	`/api/qa`	Natural language questions about agents

Events API (`/api/events`)

Method	Path	Description
GET	`/api/events/stream`	SSE stream (subscribe to real-time events)
POST	`/api/events/broadcast`	Fire an SSE event (used by norma-watch)

Alerts API (`/api/alerts`)

| GET | `/api/alerts` | Violations rendered as dashboard alerts |

Attribution API (`/api/attributions`)

| GET | `/api/attributions/{run_id}` | Failure root cause for multi-agent run |

---

16. Integration Layer (How Agents Connect)

The Core Abstraction: NormaSessionCore

Every agent integration inherits from `NormaSessionCore` (`session_core.py`). This is the base class that handles:

- Span collection (TraceCollector)
- Enforcement checks before tool calls
- Quality scoring at the end
- Trust score updates
- DB persistence (or remote POST) on context exit
- SSE broadcast (`run_completed`)

Constructor parameters:

```
NormaSessionCore(
    agent_id="my-agent",
    contract_yaml=CONTRACT_YAML,        # YAML string
    contract_version="1.0",
    db_url=None,                         # defaults to norma.db
    remote_url=None,                     # POST to server instead of DB
    parent_run_id=None,                  # for sub-agents
    initiated_by="user",                 # audit trail
    session_id=None,                     # multi-turn grouping
)
```

Circuit breaker:

- Raises `CircuitBreakerError` if `cost_so_far > max_cost_per_run` or `tool_calls > max_tool_calls` (default 50)

LangChain Adapter: NormaAgentSession

`integrations/session.py` — wraps LangChain/LangGraph agents.

Key method: `wrap_tools(tools)`

- Takes a list of LangChain `BaseTool` objects
- Monkey-patches `BaseTool._run` to insert enforcement before execution
- Returns the same tools (now wrapped) — pass these to `AgentExecutor`

```
# How an agent uses it
with NormaAgentSession("agent-id", CONTRACT_YAML) as sess:
    wrapped_tools = sess.wrap_tools(ALL_TOOLS)
    agent_executor = AgentExecutor(agent=agent, tools=wrapped_tools)
    result = agent_executor.invoke({"input": task})
    sess.record_quality(0.88)
# On exit: Run + Spans + Violations flushed to DB
```

Other Adapters

Adapter	File	Framework
`NormaOpenAIFuncSession`	`openai_func_adapter.py`	OpenAI function-calling
`NormaOpenAIAgentSession`	`openai_agent_adapter.py`	OpenAI Agents SDK
`NormaCrewAISession`	`crewai_adapter.py`	CrewAI
`NormaAutoGenSession`	`autogen_adapter.py`	Microsoft AutoGen

Middleware

- **LangChain Callback** (`langchain_callback.py`): intercepts LangChain callbacks → span recording without code changes
- **LangGraph Hooks** (`langgraph_hooks.py`): LangGraph node execution events → span hierarchy
- **Execution Logger** (`execution_logger.py`): structured logging of execution events

Builder vs. Runner Pattern (for norma-watch)

norma-watch auto-detects which pattern an agent uses:

Builder pattern: agent exposes `build_llm_agent()` / `build_langgraph_agent()`. norma-watch calls it, wraps the tools, and owns the NormaAgentSession.

Runner pattern: agent exposes `run_agent()` and manages its own session. norma-watch injects `NORMA_REMOTE_URL` env var so the agent posts telemetry to the server.

---

17. norma-watch CLI

What it does

`norma-watch` is a CLI tool that runs any Python agent under norma monitoring. It:

1. Loads the agent's Python file dynamically
2. Detects builder vs. runner pattern
3. Ensures the agent has an active contract (auto-generates if missing)
4. Runs the agent with a real LLM
5. Records the run to DB or remote server
6. Prints a summary

Usage

```
# Basic run (local DB)
poetry run norma-watch --agent-file
agents/financial_reader/earnings_report_reader.py

# With dashboard (real-time UI updates)
poetry run norma-watch \
```

```
  --agent-file agents/financial_reader/earnings_report_reader.py \
  --remote-url http://localhost:8080

# Custom task prompt
poetry run norma-watch \
  --agent-file agents/financial_reader/earnings_report_reader.py \
  --prompt "Summarize Q3 and Q4 earnings, highlight revenue trends"

# Specific contract version
poetry run norma-watch \
  --agent-file agents/financial_reader/earnings_report_reader.py \
  --contract-version 2.0
```

run_all_agents.sh

`scripts/run_all_agents.sh` runs all 16+ agents sequentially:

```
./scripts/run_all_agents.sh                      # local DB
./scripts/run_all_agents.sh --remote             # → dashboard real-time
./scripts/run_all_agents.sh --only financial     # filter by name
./scripts/run_all_agents.sh --repeat 3           # run each N times
```

Auto-Contract Generation

If an agent has no CONTRACT_YAML and no DB contract, norma-watch:

1. (Remote mode) POSTs to `POST /api/agents/onboard` → server generates contract
2. (Local mode) Calls `generate_contract_proposal()` locally with GPT-4o or stub

---

18. Frontend Dashboard

Pages

Fleet Dashboard (`/`)

The main view. Shows:

- **Summary bar**: total agents, tier breakdown, avg quality score, total cost (30d), violations (30d)
- **Alert banners**: active violations surfaced as warnings
- **Agent grid/list**: all agents as cards (AgentCard), with live "RUNNING" indicator when active
- **Topology view**: force-directed graph of orchestrators and their sub-agents
- **Run timeline**: recent runs across fleet
- **Q&A panel**: ask questions like "why did this agent's trust drop?"
- **Sentinel Governance Logs**: Sentinel sweep results (fixed-height scrollable list + detail pane)

Mode toggle (VP / Engineer):

- **VP mode**: business-friendly names, plain English messages, simplified metrics

- **Engineer mode**: agent IDs, technical detail, full stack trace

Live updates: SSE stream refreshes agent data on `run_started`, `run_completed`, `trust_changed`, `agent_created` events. Running agents show a pulsing green "RUNNING" badge.

Agent Detail (`/agents/{id}`)

- Agent identity, trust score, tier badge, enabled/paused toggle
- Trust sparkline (historical trajectory)
- Recent runs table (quality, cost, violations, timestamp)
- Active contract viewer (YAML + plain English summary)
- Contract version history (with diffs and approval trail)
- Violations log (with acknowledge/dismiss actions)
- Metrics trends (quality, cost, latency over time) with version checkpoints
- Enhancements panel (workflow recommendations)
- Attribution panel (failure root cause)
- Context routing (multi-agent delegation view)

Run Detail (`/runs/{id}`)

- Run metadata (agent, contract version, status, cost, quality)
- SpanTree: hierarchical expandable trace
- WaterfallTimeline: Gantt-style chronological span view
- QualityBreakdownPanel: per-check scores + LLM rationale
- PromptInspector: full LLM message history (system/user/assistant/tool)
- Run tree: parent-child run structure for multi-agent

Compliance (`/compliance`)

- Standards overview (OWASP, NIST, EU AI Act)
- Per-rule findings (passed/failed, evidence)
- PDF export

Alerts (`/alerts`)

- Violation inbox with severity badges
- VP message (e.g., "The finance agent tried to access confidential executive data")
- Engineer message (e.g., "data.deny violation: read_confidential called data/confidential/exec_comp.txt")
- Acknowledge + dismiss actions

Key Components

Component	File	What it does
`AgentCard`	`AgentCard.tsx`	Mini agent summary, tier badge, trust score, pulsing running indicator
`TrustSparkline`	`TrustSparkline.tsx`	Historical trust line chart with event markers

Component	File	What it does
`SpanTree`	`SpanTree.tsx`	Hierarchical collapsible trace tree
`WaterfallTimeline`	`WaterfallTimeline.tsx`	Gantt-style span timing view
`PromptInspector`	`PromptInspector.tsx`	LLM message history viewer
`MetricsTrendCharts`	`MetricsTrendCharts.tsx`	Recharts line charts with version checkpoints
`OnboardAgentModal`	`OnboardAgentModal.tsx`	Agent registration (AST scan → tool discovery → contract)
`AgentGraph`	`AgentGraph.tsx`	Multi-agent topology visualization
`GovernanceReports`	`GovernanceReports.tsx`	Sentinel log list + markdown detail pane
`QAPanel`	`QAPanel.tsx`	Conversational Q&A interface
`AttributionPanel`	`AttributionPanel.tsx`	Failure root cause visualization
`EnhancementPanel`	`EnhancementPanel.tsx`	Recommendations with YAML snippets

19. Demo Agents

All agents are in `/agents/`. Each is a standalone Python file that can run independently or under norma-watch.

Financial Reader (`agents/financial_reader/earnings_report_reader.py`)

What it does: Reads and summarizes quarterly earnings reports. **Framework:** LangChain ReAct via `build_llm_agent()` **Tools:**

- `list_reports()` → lists files in `data/public/` ✅ ALLOWED
- `read_report(filename)` → reads a public report ✅ ALLOWED
- `read_confidential(filename)` → **BLOCKED** by data.deny in all contracts
- `send_alert(recipient, message)` → **BLOCKED** in v1.0, still denied in v2.0
- `export_to_drive(filename, content)` → **BLOCKED** in v1.0, **ALLOWED** in v2.0

Demo value: Shows enforcement in action (confidential access blocked), contract versioning (v1→v2 unlocks export), and trust progression.

Research Team Orchestrator (`agents/research_team/orchestrator.py`)

What it does: 3-node LangGraph pipeline that fetches papers, analyzes metrics, and drafts a report. **Framework:** LangGraph StateGraph **Nodes:** `fetcher` → `analyzer` → `writer` **Tools:** `list_research_papers`, `fetch_research_paper`, `search_research_by_topic`, `extract_key_metrics`, `summarize_findings`, `draft_executive_report`, `read_restricted_data` (BLOCKED) **Demo value:** Multi-agent orchestration, sub-agent spans, attribution.

Other Agents

Agent	Location	Purpose
Financial Report Agent	`agents/financial_report_agent/`	Quarterly report summarizer (similar to reader)
Research Pipeline	`agents/research_pipeline/`	Research synthesis (single-agent)
Support Triage	`agents/support_triage/`	Multi-turn ticket triage
Approval Showcase	`agents/approval_showcase/`	Human-in-the-loop approval workflow
Investment Pipeline	`agents/investment_pipeline/`	NVDA analysis + risk report (LangGraph)
Compliance Review	`agents/compliance_review/`	Document compliance review (LangGraph)
Market Research	`agents/market_research/`	Sector trend analysis
Norma Sentinel	`agents/norma_sentinel/`	Governance sweep of the entire fleet
Red Team	`agents/red_team/`	Security attack simulations (intentional violations)
Violations Showcase	`agents/violations_showcase/`	Deliberately violates policies (for demo)
OpenAI Research	`agents/openai_research/`	OpenAI Agents SDK demo
OpenAI Func	`agents/openai_func/`	OpenAI function-calling demo
Standalone OTel	`agents/standalone_otel/`	Manual span instrumentation (posts to ingest API)

Agent Data Files

```
data/public/          ← q2_2025_earnings.txt, q3_2025_earnings.txt,
q4_2025_earnings.txt
data/confidential/    ← exec_compensation_2025.txt (BLOCKED by all
contracts)
data/research/        ← AI research papers
data/compliance/      ← compliance documents
data/support/         ← support KB articles
```

---

20. Real-Time Events (SSE)

What is SSE?

Server-Sent Events (SSE) is a browser standard for server-push updates over a persistent HTTP connection. The browser opens `GET /api/events/stream` once and receives a stream of JSON

messages whenever something happens.

How norma uses SSE

The backend has an in-process event bus (`_subscribers: list[asyncio.Queue]`). When `broadcast(event_type, data)` is called, it pushes to all connected queues.

Event Types

Event	When fired	Data
`run_started`	When agent execution begins	`{agent_id, mode}`
`run_completed`	When a run finishes	`{agent_id, blocked, run_id}`
`trust_changed`	When trust score updates	`{agent_id, new_score}`
`violation_detected`	When enforcement blocks a tool	`{agent_id, policy_rule}`
`agent_created`	When an agent is onboarded	`{agent_id, name, tier}`
`agent_paused`	When an agent is disabled	`{agent_id, enabled: false}`
`agent_resumed`	When an agent is re-enabled	`{agent_id, enabled: true}`

Frontend hook

```
const { lastEvent } = useEventStream();
useEffect(() => {
  if (lastEvent?.type === "run_started") {
    setRunningAgents(prev => new Set([...prev, lastEvent.data.agent_id]));
  }
  if (lastEvent?.type === "run_completed") {
    setRunningAgents(prev => { const s = new Set(prev);
s.delete(lastEvent.data.agent_id); return s; });
    refreshAgents();
  }
}, [lastEvent]);
```

External broadcast

`POST /api/events/broadcast {type, ...data}` — lets norma-watch CLI fire events to the dashboard without being inside the FastAPI process.

---

21. Multi-Agent Orchestration

How norma handles orchestrators

An orchestrator is an agent that delegates work to sub-agents. norma models this as a **run tree**:

- Parent run: the orchestrator's execution

- Child runs: each sub-agent execution (linked via `parent_run_id`)
- `initiated_by` on child runs = `"orchestrator:{parent_agent_id}"`

Span hierarchy in multi-agent runs

```
session span (root)
  └── llm_call span (orchestrator decides to delegate)
      └── agent_handoff span (norma records delegation)
          └── tool_call span (sub-agent calls a tool)
          └── llm_call span (sub-agent calls LLM)
```

`push_subagent_span()` / `pop_subagent_span()` in session_core create the `agent_handoff` spans.

Virtual sub-agents

For LangGraph agents, norma parses `.add_node()` calls via `_workflow_stage_names_from_file()` and creates virtual Agent records for each node. These appear as sub-agents in the topology view even before the graph runs.

Context routing

When an orchestrator delegates, `route_context()` in `context_router.py` applies the delegation rules from the parent's contract:

```yaml
delegation:
  subagents:
    - id: fetcher-agent
      context_limit_tokens: 2000
```

Context metrics are recorded per delegation to track token utilization.

---

22. Contract Lifecycle (End-to-End)

```
1. ONBOARD
   User pastes directory path into OnboardAgentModal
   → Backend AST-scans Python files (introspect.py)
   → Discovers @tool-decorated functions
   → Calls contract_generator.py (LLM or stub)
   → Creates Agent record (tier=restricted, trust=0.40)
   → Creates Contract record (is_active=False initially)
   → Auto-approves for demo (auto_approve=True flag)
   → SSE: agent_created event

2. AGENT RUNS (restricted tier)
   → enforcement.py enforces contract rules
   → trust_engine records clean_run → +0.025 per run
```

```
   → After 10 clean runs at ≥0.65 score:

3. TIER PROMOTION PROPOSAL
   → trust_engine._check_tier_upgrade() fires
   → Sets Agent.pending_contract_version = "2.0"
   → Creates new Contract record (is_active=False)
   → Dashboard shows "Tier Promotion Available" banner

4. HUMAN APPROVAL
   → User clicks "Approve" in dashboard
   → POST /api/contracts/{agent_id}/activate
   → Old contract: is_active=False
   → New contract: is_active=True, activated_at=now
   → Agent.current_tier = "standard"
   → SSE: trust_changed event

5. AGENT RUNS (standard tier)
   → New contract enforced
   → If violation: trust drops, tier reverts to restricted
   → Repeat from step 2 for trusted tier
```

---

23. Agent Execution Flow (End-to-End)

```
User clicks "Run" in dashboard
        │
        ▼
POST /api/agents/{id}/execute  (agents.py)
        │
        ├── Load agent module (dynamic import)
        ├── Load active contract from DB
        ├── Discover tools (@tool decorated functions)
        │
        ├── broadcast("run_started", {agent_id})  ──► SSE ──► UI shows
"RUNNING" badge
        │
        ├── Build task plan (tool sequence)
        │
        ├── NormaAgentSession(agent_id, contract_yaml).__enter__()
        │       └── TraceCollector initialized
        │       └── Session span started
        │
        ├── For each tool call:
        │       ├── enforce(ExecutionContext) ← enforcement.py
        │       │       ├── Check tool ACL
        │       │       ├── Check data paths
        │       │       ├── Check output patterns
        │       │       ├── Check cost SLA
        │       │       └── Check latency SLA
        │       │
        │       ├── BLOCKED → record violation, return block message
```

```
          │       └── ALLOWED → execute tool, record tool_call span
          │
          ├── LLM call → record llm_call span (tokens, cost, model)
          │
          ├── NormaAgentSession.__exit__()
          │       ├── Quality scoring (quality_scorer.py)
          │       ├── trust_engine.record_clean_run() or record_violation()
          │       ├── Flush Run + Spans + Violations to DB
          │       └── broadcast("run_completed", {agent_id})  ──► SSE ──► UI
  refreshes
          │
          └── Return result JSON to frontend
```

---

24. Compliance Posture

What "compliance posture" means

An agent's compliance posture is the summary of how it fares against OWASP LLM Top 10, NIST AI RMF, and EU AI Act rules. Shown as CMP PASS / CMP FAIL badge on every AgentCard.

How it's computed

`GET /api/compliance/{agent_id}/posture`:

1. Load agent's recent runs and spans from DB
2. Load active contract
3. Pass all context to `ComplianceEngine.evaluate(ctx)`
4. Aggregate: passed = all 12 rules passed; failed = any rule failed

PDF Export

`GET /api/compliance/{agent_id}/export/pdf` generates a downloadable PDF report using `fpdf2`. Includes findings, evidence, and recommendations per standard.

---

25. Q&A Engine

What it is

A natural language interface to query norma's data. Ask questions like:

- "Why did financial-reader's trust drop?"
- "Which agent has the highest cost this week?"
- "Show me all violations in the last 30 days"
- "Is research-team compliant?"

How it works

`POST /api/qa {question, agent_id?}`

1. **Intent classifier**: regex patterns match the question to one of: `trust_drop`, `trust_rise`, `violations`, `cost`, `quality`, `fleet`, `runs`, `latency`, `compliance`
2. **Handler**: pulls relevant data from DB (runs, violations, spans, trust history)
3. **Answer construction**: deterministic data-driven response
4. **LLM fallback**: if OPENAI_API_KEY set and intent unclear, GPT-4o gets the DB context + question
5. **Response fields**: `answer`, `data_sources` (what tables/runs were queried), `confidence` (0.0–1.0), `caveats`

---

26. Feature Flags

All in `backend/norma/config.py` via Pydantic settings:

Flag	Default	Effect
`enable_llm_quality_scoring`	`True`	Uses GPT-4o-mini as quality judge
`enable_semantic_enforcement`	`False`	Placeholder; semantic (LLM-based) enforcement not implemented
`enable_otlp_export`	`False`	Export spans to external OTel collector
`enable_api_key_auth`	`False`	Require API key header (Phase 7)
`enable_webhooks`	`False`	Outbound webhooks on events (Phase 7)

Set via environment variables: `ENABLE_LLM_QUALITY_SCORING=false`.

---

27. Security

Enforcement is the security model

The enforcement engine (5 checks) is the primary security mechanism. It is:

- Deterministic (no LLM = no hallucination)
- Pre-execution (tools cannot run before enforcement passes)
- Always on (no per-call bypass)

PII / Credential Detection

Output patterns check for:

- Credit card numbers (13–16 digit sequences)
- SSNs (XXX-XX-XXXX format)
- PII name patterns (Title Case name matches)
- Credentials (`password=...`, `api_key=...`, `secret=...`, `token=...`)

Contract = least privilege

Every contract defaults to deny-all for unlisted tools and data paths. Allowed lists must be explicit.

Human oversight is mandatory

No contract auto-activates. No tier promotion happens automatically. Every privilege increase requires a human approval event in the audit trail.

Audit trail

Every contract change, approval, violation, and review decision is timestamped and attributed to an actor in the database.

API auth (planned)

`enable_api_key_auth=True` enables API key validation with role-based access (viewer / operator / admin). Not yet active.

---

28. File Map (Quick Reference)

Backend Core Engines

File	Purpose
`norma/core/trust_engine.py`	Trust scoring, tier transitions
`norma/core/enforcement.py`	Pre-execution policy checks (5 checks)
`norma/core/quality_scorer.py`	Deterministic + LLM quality evaluation
`norma/core/contract_engine.py`	YAML parsing, validation, summaries
`norma/core/contract_generator.py`	Auto-generate proposals (never auto-activate)
`norma/core/attribution.py`	Failure root cause analysis
`norma/core/enhancement.py`	Workflow improvement recommendations
`norma/core/context_router.py`	Context budget enforcement
`norma/core/trace.py`	OTel-compatible span collection
`norma/core/pricing.py`	LLM cost calculation
`norma/core/compliance/engine.py`	Runs all compliance rules
`norma/core/compliance/owasp.py`	OWASP LLM Top 10 rules
`norma/core/compliance/nist.py`	NIST AI RMF rules
`norma/core/compliance/eu_ai_act.py`	EU AI Act rules
`norma/core/compliance/drift.py`	Model drift detection

Backend API

File	Purpose
`norma/api/agents.py`	Fleet, onboarding, execution (largest file)

File	Purpose
`norma/api/runs.py`	Run telemetry, spans, ingest
`norma/api/violations.py`	Enforcement audit log
`norma/api/alerts.py`	Violations as dashboard alerts
`norma/api/contracts.py`	Version history, generation, approval
`norma/api/compliance.py`	Compliance evaluation, PDF export
`norma/api/qa.py`	Natural language Q&A
`norma/api/analytics.py`	Metrics, trends, version checkpoints
`norma/api/attributions.py`	Failure root cause
`norma/api/events.py`	SSE real-time events + broadcast

Backend Integrations

File	Purpose
`norma/integrations/session_core.py`	Framework-agnostic base class
`norma/integrations/session.py`	LangChain/LangGraph adapter
`norma/integrations/openai_func_adapter.py`	OpenAI function-calling
`norma/integrations/openai_agent_adapter.py`	OpenAI Agents SDK
`norma/integrations/crewai_adapter.py`	CrewAI
`norma/integrations/autogen_adapter.py`	AutoGen
`norma/integrations/watch.py`	`norma-watch` CLI entry
`norma/agents/introspect.py`	AST tool discovery, onboarding

Frontend

File	Purpose
`app/page.tsx`	Fleet dashboard
`app/agents/[id]/page.tsx`	Agent detail
`app/runs/[id]/page.tsx`	Run detail + span tree
`app/compliance/page.tsx`	Compliance posture
`app/alerts/page.tsx`	Alert inbox
`hooks/useEventStream.ts`	SSE hook
`hooks/useMode.ts`	VP/Engineer mode

File	Purpose
`lib/api.ts`	API client functions
`lib/types.ts`	TypeScript interfaces
`components/AgentCard.tsx`	Agent summary card
`components/MetricsTrendCharts.tsx`	Time-series charts
`components/GovernanceReports.tsx`	Sentinel log list
`components/SpanTree.tsx`	Hierarchical trace view

---

29. What's Real vs. Planned

Fully Implemented & Tested

- Trust engine (score math, tier transitions, clean run counter)
- Enforcement (all 5 checks, deterministic, always runs)
- Quality scoring (deterministic checks + LLM judge)
- Contract YAML parsing, validation, versioning, approval
- Contract auto-generation (LLM + stub)
- Compliance rules (OWASP 5 + NIST 3 + EU AI Act 3 + drift)
- Attribution engine (probabilistic, multi-node)
- Enhancement engine (3 recommendation types)
- Context routing (token budget enforcement)
- Span system (custom OTel-compatible, persisted to DB)
- All API endpoints (40+ routes)
- All UI pages and components
- SSE real-time events
- norma-watch CLI (any agent, builder or runner)
- Multi-agent orchestration (run tree, sub-agent spans)
- PDF compliance export
- Q&A engine (intent-based + LLM fallback)
- Agent onboarding via AST scan
- Code change detection (file hash)
- 97 passing tests

Approximated / Partial

- Token counts in scripted mode = 0 (correct behavior; real tokens only in LLM mode)
- Cost estimates use pricing catalog (directional, not billing-accurate)
- Context overlap ratio = n-gram similarity (not semantic similarity)
- NIST and EU AI Act rules are basic placeholder checks (not deep compliance audits)
- Multi-turn conversation memory is TTL-based key-value, not semantic memory

Not Yet Built

- `enable_semantic_enforcement` — flag exists, no LLM-based enforcement

- `enable_otlp_export` — flag exists, OTLP export not wired
- `enable_api_key_auth` — flag exists, auth middleware is a stub
- `enable_webhooks` — flag exists, no outbound webhooks
- WebSocket streaming (using SSE + polling instead)
- Semantic search over run history

---

30. Common Questions & Answers

Q: Does norma change the agent's code? A: No. The agent's LLM logic is untouched. norma wraps the tool functions (monkey-patches `BaseTool._run`) and the session context. The agent runs normally; norma intercepts at the tool boundary.

Q: What happens if the backend is down and the agent still runs? A: For builder-pattern agents (using NormaAgentSession), the session writes to the local SQLite DB directly, so it still records. For runner-pattern agents that POST to the remote API, runs won't be recorded if the server is unreachable.

Q: Can an agent bypass enforcement? A: Not through normal operation. Enforcement is applied by the session wrapper before any tool executes. An agent would have to directly call the underlying tool function (bypassing `BaseTool._run`) to escape it, which is not how LangChain agents work.

Q: What does "restricted tier" mean practically? A: An agent at the restricted tier has the most locked-down contract. Fewer tools are allowed, data access is minimal, and SLA limits are tighter. As it earns trust through clean runs, the contract can be upgraded (with human approval) to standard or trusted, unlocking more capabilities.

Q: How are costs calculated? A: From LLM provider token counts × pricing per million tokens from `pricing_models.yaml`. The catalog has prices for GPT-4o, GPT-4o-mini, GPT-3.5-turbo, Claude models, Gemini, etc.

Q: What is the quality score based on? A: Four deterministic checks (output length, absence of error keywords, format compliance, absence of PII/credential patterns) plus an optional GPT-4o-mini judge. Composite = 40% deterministic + 60% LLM if both enabled.

Q: Why does trust never auto-promote? A: By design. norma's philosophy is that AI agents should not grant themselves more privileges. A human must review the proposed new contract and explicitly approve it. This is the governance guarantee.

Q: What is a span vs. a run? A: A run is the full execution of an agent from start to finish (one record). A span is one operation within that run (LLM call, tool call, etc.). One run = many spans. Spans form a trace tree.

Q: How is norma different from LangSmith / Weave? A: LangSmith and Weave are observability tools — they record what happened. norma adds enforcement (prevents things from happening) and governance (contracts, human approval, trust progression). norma also does compliance checking against OWASP/NIST/EU AI Act, which observability tools don't.

Q: How does the onboarding modal work? A: You paste the path to an agent directory. norma's AST scanner reads the Python files, finds all `@tool`-decorated functions, lists them as discovered tools,

generates a contract proposal (using GPT-4o if API key available), and presents it for review. Approval activates it.

Q: What is the Sentinel agent? A: `agents/norma_sentinel/sentinel.py` is an agent that sweeps the entire fleet — checks all agents' recent runs, flags anomalies, and writes a governance report (stored in `compliance_reports` table, shown in GovernanceReports panel on the dashboard).

Q: Can norma monitor agents from different frameworks in the same fleet? A: Yes. There are adapters for LangChain, LangGraph, OpenAI function-calling, OpenAI Agents SDK, CrewAI, and AutoGen. They all inherit from `NormaSessionCore` and produce the same Run/Span records.

Q: How does the version checkpoint on the trend chart work? A: The analytics API queries contract change timestamps and span model names (to detect when the LLM model changed). These are returned as `version_checkpoints` with `change_type: "contract"` (purple line) or `change_type: "model"` (orange line, labeled with model name like "gpt-4o").

---

31. Additional Core Modules (Coverage Check)

Anomaly Detector (`core/anomaly_detector.py`)

Detects statistical outliers in cost, quality, and error rates across rolling time windows.

Every alert includes:

- The metric and its current value
- The baseline (previous window) value
- Sample sizes for both windows (`current_n`, `baseline_n`)
- Whether any contract or model change was recorded in the window (flags confounders)
- What the data does **not** tell us (norma never overclaims)
- A suggested next action

Output: `AnomalyAlert` dataclass with `metric`, `current_value`, `baseline_value`, `change_pct`, `current_window`, `baseline_window`, `contract_change_in_window`, `model_change_in_window`, `vp_message`, `engineer_message`, `severity`.

This powers the Alerts API — violations and anomalies are surfaced as alerts with both VP-friendly and engineer-friendly messages.

Drift Detector (`core/drift_detector.py`)

Detects behavioral drift across rolling time windows by comparing:

- Quality score distributions (mean, stdev)
- Cost per run
- Tool call frequencies (which tools are being called more/less)
- Prompt hash distribution (input drift — detects if prompts are changing)

Flags drift when any metric shifts > 2σ from baseline. Returns `DriftEvent` objects with:

- `drift_type`: `quality_drift` | `cost_drift` | `input_drift` | `tool_frequency_drift`

- `severity`: `warning` | `critical`
- `baseline_value`, `current_value`

Different from the compliance `ModelDriftRule` — drift_detector looks at behavioral/statistical changes, not just model name changes.

OTel Export (`core/otel_export.py`)

Converts norma's internal `SpanData` objects to the OpenTelemetry Protocol (OTLP) JSON format and POSTs them to an external OTel collector (Jaeger, Grafana Tempo, etc.).

- Converts timestamps to nanoseconds (OTel standard)
- Generates proper `trace_id` (SHA-256 of `agent_id:run_id`)
- Maps span attributes to OTLP `KeyValue` pairs
- Uses `httpx` to POST to `settings.otlp_endpoint`
- Controlled by `enable_otlp_export` feature flag (currently `False` by default)

This is what would allow norma spans to appear in external observability platforms alongside other system traces.

Version Comparator (`core/version_comparator.py`)

Compares metrics between runs under two different contract versions (before/after a contract change).

Takes two lists of run dicts (segmented by contract version) and returns:

- `quality_delta`, `cost_delta`, `completion_delta`, `latency_delta`
- `n_v1`, `n_v2` (sample sizes for each version)
- Statistical noise warning if sample sizes are too small to be conclusive

Used by the analytics API when computing version checkpoints and contract comparison views.

Q&A Engine (`core/qa_engine.py`)

The backend implementation of the conversational Q&A system. Separate from `api/qa.py` (which handles routing) — the engine handles:

- Structured query construction from intent
- Data retrieval (runs, violations, spans) scoped to the question
- LLM call construction (system prompt + JSON context)
- Response parsing with confidence and caveats

Design rules baked into the system prompt:

> *Only state what the data supports. Always include sample size (n=X runs) and time window. Never speculate about causes that cannot be confirmed from run data. End by stating what cannot be determined.*

Confidence levels: `high` | `medium` | `low` | `cannot_determine`

Webhooks (`core/webhooks.py`)

Outbound webhook delivery for operational notifications to:

- **Slack** (`settings.webhook_slack_url`)
- **Email** (`settings.webhook_email_url`)
- **PagerDuty** (`settings.webhook_pagerduty_url`)

Uses `httpx` for HTTP delivery. Controlled by `enable_webhooks` feature flag (currently `False`). When enabled, significant events (tier revocation, repeated violations, quality drops) would trigger notifications.

Telemetry API (`api/telemetry.py`)

A dedicated telemetry ingest endpoint, separate from `runs.py`'s `/ingest`. Designed for external agents that want to push spans directly via HTTP without using the norma SDK:

```
POST /api/telemetry/ingest
```

Accepts the same payload shape as `runs/ingest` (agent_id, framework, spans, violations, run_status, etc.) plus optional `X-Norma-Key` header for API key auth.

This is what the `standalone_otel` demo agent uses — it constructs span data manually and POSTs it, demonstrating that norma can receive traces from agents that don't use the Python SDK at all.

Support Agent (`agents/support_agent/`)

A multi-turn support ticket triage agent (separate from `support_triage`). Handles escalation workflows and human handoff scenarios. Used to demonstrate the `session_id` grouping (multiple turns in one conversation) and the `escalated` completion status.

`run_all_agents.sh` Coverage

All agents in `agents/` are listed in `scripts/run_all_agents.sh`. Contract auto-generation handles agents without `CONTRACT_YAML`. The `--only` filter runs a subset (e.g., `--only financial` to run only financial-related agents).